Chapter 9

Simulating Dynamical Processes on Networks

In previous chapters we discussed complex network models where the structure of the graph followed a certain distribution. Alternatively, one might fix the structure of a graph to, say $G = (V, E)$, and simulate and study a process on the network. The aim of this chapter is two-fold: Firstly the aim will be to study simulation and visualization techniques for dynamical states of networks and secondly the aim will be to deepen our knowledge on the dynamics of processes on networks by looking on processes on networks of finite size.

Given a graph $G = (V, E)$ it is possible to assign a weight or parameters to the vertices and/or to the edges of that graph. We will call these weights the *states of the vertices* and/or *states of the edges*, if they can change over time. The state of the graph is the entirety of the states of its nodes at a certain point of time.

Think of, for instance, networks in epidemics where a vertex can get infected by a contact with a neighboring infected vertex. Here the *state of the vertex* could be modeled by a variable with values in $\{0, 1\}$, where 0 stands for susceptible (but not-infected) and 1 stands for infected. The state of the graph would be given by a vector in $\{0, 1\}^{|V|}$.

An example for network models where edges are augmented with a state could be imagined in for the simulation of traffic networks, where the density of traffic on a street is modelled as a variable in $\mathbb{R}_{\geq 0}$. In this case the state of the traffic network would be given by a vector in $\mathbb{R}^{|E|}$.

9.1 Examples of dynamical processes on networks

Let us look at some of dynamical processes on graphs and their states in more detail.

Epidemiological Processes In epidemiology it is studied how a disease spreads across a population. Epidemiological models have applications also in the analysis of computer networks, for instance to study the spread of messages and computer viruses through the internet. An example would be the *spread of a virus through a network*. The virus can only be transmitted through contact across adjacent nodes. A node can be susceptible to a disease or infected. It might also be the case that a

node gets immunized or is removed from the network some time after infection. Instead of removing it, one could set its state to inactive. The common terminology in epidemiology is like this:

- SI model: A node can get infected after being in a susceptible state. Once a node is infected it remains being infected and it can infect other nodes. Such dynamics occur also in the spread of messages and information across computer networks.
- SIR model: In this model, an infected node can be recovered after some time of the infection from this spread and can be considered removed from the network. Reasons for this could be that the infected individual got immunized, or the infected individual got isolated from the population, or the disease was lethal.
- SIS model: Here nodes that are infected can recover and return to the susceptible state. After this they might get infected again.

The modeling and simulation of epidemiological process will be in the focus of attention in the examples discussed in this chapter.

Depending on the level of detail of the model, different sets of state variables could be chosen. The most simple choice is to model the state of the vertex by means of a binary variable, one variable assigned to each node. In order to model the recovery process, one might use an additional state variable for each node, that stands for the time that has passed since infection and it is set to zero for susceptible vertices. In some models a weight is assigned to the edges of the network, symbolizing for instance the strength of the protection to virus attacks. If this weight does not change over time, we will not treat it as a state variable but treat it as a constant.

*__Transport and diffusion processes__ In *transport phenomena* the state variables might be the number or concentration of particles located at a vertex. And the particles can diffuse through the network. The edge weights can model the diffusion constants and length of the connections and they are typically constant.

Some fundamental properties when simulating transportation dynamics are:

- Particles can move from one node to another via an edge of the graph. In such a case the weight (particle count) of the first node would be decreased and the weight of the destination node would be increased.
- There are designated nodes that introduce new particles to the network (source nodes), and that remove particles from the network (sink nodes).

*__Energy fields and molecular simulation__ In the simulation of molecular structures and the behavior of crystals, the network is given by the arrangement of atoms, for instance in a crystal structure. Neighboring atoms can influence each others state and typically these interactions are partly random and partly driven by the local minimization of free energy.

*__Pulse-coupled Oscillators__ In order to understand robust mechanisms for the synchronizations of nodes in a network, the model of pulse coupled oscillators is widely used. The charge of a vertex is modeled as a continuous state variable. The charge

increases over time until it reaches a threshold. Then a sudden discharge takes place that results in a signal (e.g. an electrical pulse) and a discontinuous change of charges in the neighboring vertices. Under certain settings, such processes can lead to robust synchronization and have been used to explain the synchronization of the pacemaker cells in the sinusidal node of the heart or to design energy efficient sensor networks.

*__Reaction diffusion systems__ Reaction diffusion systems occur in biological processes. Their study has led to biological models that can be used to understand the emergence of natural patterns, such vegetation patterns or animal skin patterns. In *reaction diffusion* systems, the state variables are the concentrations (state variables) of different chemical substances at vertices. The dynamics is governed by the interplay of diffusion of these substances across the networks and chemical conversions (reactions) occuring at the vertices.

A prominent application of reaction diffusion systems was provided by Alan Turing, who proposed reaction diffusion model as a model that can explain the emergence of the spots on the leopards' skin. Alan Turing stated the laws that govern the systems dynamics in the form of partial differential equations, but was not able to derive closed form expressions for the resulting phenomena. He remarked that even if we cannot capture such phenomena in closed form mathematical expressions, we can still study them by means of computer simulations.

9.2 Visualization of dynamic state using NetworkX

Next, some techniques for simulating dynamical processes on graphs will be discussed. In order to do so, it will first be discussed, how to manage and visualize states of a graph using the NetworkX package.

9.2.1 Representing and visualizing states in NetworkX

The following steps can be taken to draw and to update a graph and the states of its nodes in `NetworkX`.

First we create a random graph with 10 nodes:

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

G = nx.erdos_renyi_graph(10, 0.4, directed=False)
```

Next we define the states at the graphs nodes:

```
for each in G.nodes():
  G.nodes[each]['state'] = 0
```

and plot the graph:

```
vertex_color = [G.nodes[v]['state']*1 for v in G.nodes()]

plt.figure(figsize=(10, 10))
nx.draw(G, node_color=vertex_color, with_labels=True)
plt.show()
```

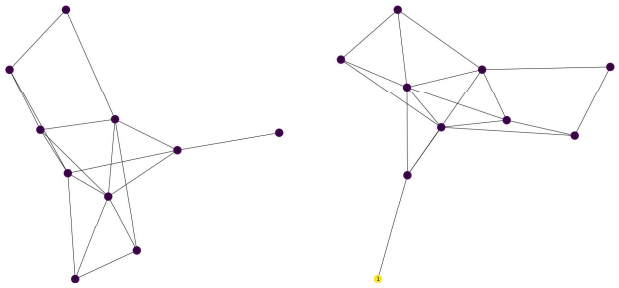


Figure 9.1: Visualization of random graph where nodes are colored based on their states. In the first graph, all nodes have same state and in the second graph, node 1 has a different state.

Next, we update the state of node 1:

```
G.nodes[1]['state'] = 1
```

and plot the graph:

```
vertex_color = [G.nodes[v]['state']*1 for v in G.nodes()]

plt.figure(figsize=(10, 10))
nx.draw(G, node_color=vertex_color, with_labels=True)
plt.show()
```

Both of these graphs are shown in Figure 9.1, respectively.

We also can assign weights to the edges of a graph. In the following code, we assign the weight of each edge as the multiplication of the degree of its end nodes. Then we plot the graph where the width of each edge is proportional to its weight, as shown in Figure 9.2.

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

G = nx.erdos_renyi_graph(10, 0.4, directed=False)

for each in G.edges():
  G[each[0]][each[1]]['weight'] = G.degree(each[0])*G.degree(each[1])

edge_width = [G[each[0]][each[1]]['weight'] for each in G.edges()]

plt.figure(figsize=(10, 10))
nx.draw(G,  width=edge_width, with_labels=True)
plt.show()
```

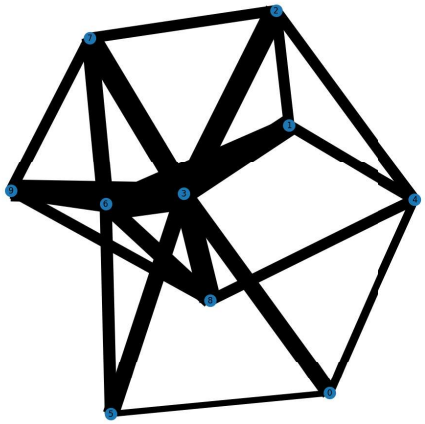


Figure 9.2: Visualization of ring graph with continuous edge weights and discrete node weights.

9.3 Simulation methods

In the following we are going to study different simulation and visualization techniques for dynamics on complex networks.

Our focus will be on the simulation of the dynamics discrete states on networks and examples are provided below. The introduction will be by means of programming examples and we will focus on examples from epidemiology – but starting from these examples it will in many cases be straightforward to develop simulators for other dynamic processes on graphs, too.

- Deterministic simulation
  - Cellular automata (local update rule), agent based models
  - Differential equations (continuous time, continuous change rates)
- Stochastic simulation
  - Monte-carlo simulation (simulation using random numbers, a different trajectory is produced in every run)
  - Simulation of hold-and-jump processes: System undergoes discrete changes in a countable set of points in time. For instance, Stochastic Simulation Algorithm (SSA) of Continuous Time Markov Chains
  - Simultation of stochastic distribution (exact probabilities for system being in a particular state over time, e.g., using Time-Homogeneous Markov Chains)
  - Mean field simulation (studies the dynamics of aggregated states on large state spaces)

Typically, there will be a trade-off between the simplicity of the program and its computational efficiency. Some examples will be provided for how to make simulation programs faster by introducing auxilliary data structures. Often the speed-up will be significant.

9.4 Discrete time simulation in `NetworkX/Python`

In discrete, time-homogeneous simulation, the state of the network is updated at discrete time steps $t, t \in \mathbb{N}_0$. Typically the state of the system at time step $t = 0$ is given, and from the state of the system in time step t the state of the system in time step $t+1$ is computed by means of a *state transition function*.

In most application, the assumption that events can only happen at discrete time steps is a simplification. If the time steps are very small the simulation tends to be more realistic but the simulation time is very long. On the other hand, if time steps are too big, the simulation tends to become unrealistic due to the coarse-grained model. Our discussion of discrete time simulation shall serve as a bridge to a more suitable method of simulating dynamical processes on complex networks, CTMC models, that we will discuss in detail in the subsequent chapter.

In NetworkX, we will augment the graph with its state variables, so that we can compute a single step of the simulation by means of the following recursion, which updates a network `G` in some time step to the network `G` in the subsequent time step, by means of the recursion. In the following example, at a given time, each node is either suspicious or infected. In the next iteration, the state of each node is updated to infected if any of its node is infected. To do so, we first create a list of nodes called newly_infected_nodes and then set their state to “i".

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

G = nx.erdos_renyi_graph(10, 0.4, directed=False)

for each in G.nodes():
  G.nodes[each]['state'] = 's'

#set that node 1 is infected that will spread the infection to the network.
G.nodes[1]['state'] = 'i'

newly_infected_nodes=[]
for each in G.nodes():
  if G.nodes[each]['state'] == 'i':
    for ngh in G.neighbors(each):
      newly_infected_nodes.append(ngh)

#Now update the state of each newly infected node to "i"
for each in newly_infected_nodes:
    G.nodes[each]['state'] = 'i'
```

The transition function that you implement can be deterministic or can also introduce random decisions in case of stochastic system simulation. In order to simulate

N time steps we can use a loop, and repeat the above process.

9.5 Percolation Theory

A percolation process involves randomly marking the vertices or edges of a graph as either “occupied" or “unoccupied," and then analyzing the resulting vertex patterns for specific properties. Initially proposed in the 1950s, the percolation model was primarily motivated by the need to simulate disease spread. We will start our discussion with its simple application to the question of network resilience.

Real-world networks are often highly resilient to random vertex deletions. Resilience can be assessed in various ways, with the simplest being the stability of the fraction of vertices in the largest, or “giant," component of the network. In communication networks, for instance, vertices in the giant component can communicate with a significant portion of the network, while those in smaller components are limited to minimal connections. The issue of resilience to random vertex failures is equivalent to site percolation on the network, while edge failures relate to bond percolation. Vertices are randomly occupied (functioning) or unoccupied (failed), and the number of vertices remaining that can successfully communicate is precisely the giant component of the corresponding percolation model.

The removal of a single node typically has minimal impact on a network's overall integrity (as shown in Figure 9.3). However, removing multiple nodes can fragment the network into isolated components (Image 8.3d). Naturally, the more nodes removed, the higher the likelihood of network fragmentation. This leads to the critical question: how many nodes must be deleted to break a network into disconnected parts? For instance, how many Internet routers need to fail for the Internet to split into isolated clusters unable to communicate? To explore this, we turn to the mathematical framework of network robustness provided by percolation theory.

Percolation theory is a well-established subfield of statistical physics and mathematics. It addresses problems like the one shown in Figure 9.4. Consider a square lattice where pebbles are randomly placed at each intersection with probability some p, and neighboring pebbles are considered connected and they start forming clusters of size two or more. Since the placement of each pebble is random, the key question is:

- What is the expected size of the largest cluster?
- What is the average cluster size?

In general, the higher is p, the larger are the clusters. An interesting result in percolation theory is that the cluster size does not change gradually with p. Rather, for a wide range of p, the lattice contains many small clusters (as shown in Figure 9.4). However, when p reaches to a critical threshold p_c, these small clusters grow and merge, forming a large cluster, called “percolating" cluster as it reaches the end of the lattice. In other words, at p_c we observe a phase transition as this cluster spans the entire lattice from numerous small clusters to a percolating cluster that percolates the whole lattice.

As shown in Figure 9.4 (a), for small p most pebbles are isolated, and the largest cluster has only three nodes, highlighted in purple. In Figure 9.4 (b), for large p most of the pebbles belong to a single large cluster (colored purple), i.e., the percolating cluster, as it spans the whole lattice.

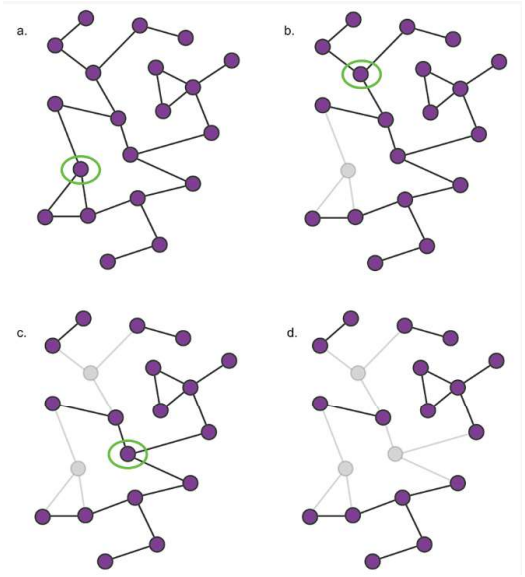


Figure 9.3: The gradual fragmentation on a sample network following the breakdown of its nodes; figure is taken from [22]

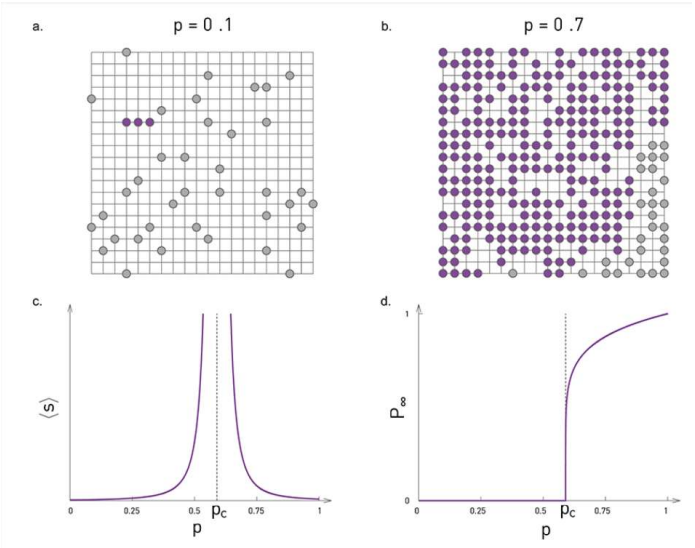


Figure 9.4: An example of percolation theory where we randomly place pebbles with probability p on a square lattice.; figure is taken from [22]

We focus on three main quantities to analyze this phase transition:

1. Average Cluster Size $\langle s \rangle$: According to percolation theory the average size of all finite clusters follows:

$\langle s \rangle = |p - p_c|^{-\gamma_p}$

The average cluster size diverges as we approach p_c. In Figure 9.4 (c), The average cluster size, $\langle s \rangle$, increases as p approaches p_c from below, as small clusters begin to merge. At p_c, $\langle s \rangle$ diverges, following the above Equation. Above p_c, the same divergence occurs, but $\langle s \rangle$ is calculated excluding the percolating cluster. The divergence on both sides of the critical point is characterized by the same exponent, γ^p.

2. Order Parameter (P_∞): The probability P_∞ that a randomly chosen pebble belongs to the largest cluster follows:

   $P_\infty \sim (p - p_c)^{\beta_p}$

   Therefore as p decreases towards p_c the probability that a pebble belongs to the largest cluster drops zero. In Figure 9.4 (d), we show this through an example. For $p < p_c$ all components are small, so P_∞ is zero. Once p reaches p_c a giant component emerges, and after that there is a finite probability that a node belongs to the largest component.

3. Correlation Length (ξ): The mean distance between two pebbles that belong to the same cluster follows:

   $\xi \sim |p - p_c|^{-\nu}$

   When $p < p_c$ the distance between the pebbles in the same cluster is finite, and at p_c this distance diverges. This means that at p_c, the size of the largest cluster becomes infinite, allowing it to percolate the whole lattice.

Just note that the value of p_c depends on the lattice type (square, triangular or hexagonal lattice) and its dimension.

9.6 Network Robustness and Inverse Percolation Transition

In network robustness, the primary concern is how node failures affect a network's integrity. Percolation theory helps describe this process. Consider a square lattice as a network where nodes are intersections (Figure 9.5). We randomly remove a fraction f of nodes and assess how their absence impacts the network's structure. When f is small, the network remains largely intact. As f increases, more nodes are isolated from the giant component. Eventually, with sufficiently large f, the giant component fragments into small disconnected components as shown in Figure 9.5.

The fragmentation process is not gradual but defined by a critical threshold f_c. For $f < f_c$, a giant component persists, but when f exceeds f_c, the giant component disappears. This is demonstrated by the dependence of p_∞ on f, where p_∞ represents the probability of a node being part of the giant component. p_∞ remains non-zero for $f < f_c$, but drops to zero as f_c is approached. The critical exponents governing this breakdown, γ_p, β_p, and ν, are the same as we discussed in the last section, and these two processes can be mapped to each other by choosing $f = 1 - p$.

However, if the underlying network is not as regular as a square lattice, the answer for network failure depends on the network topology. In summary, network

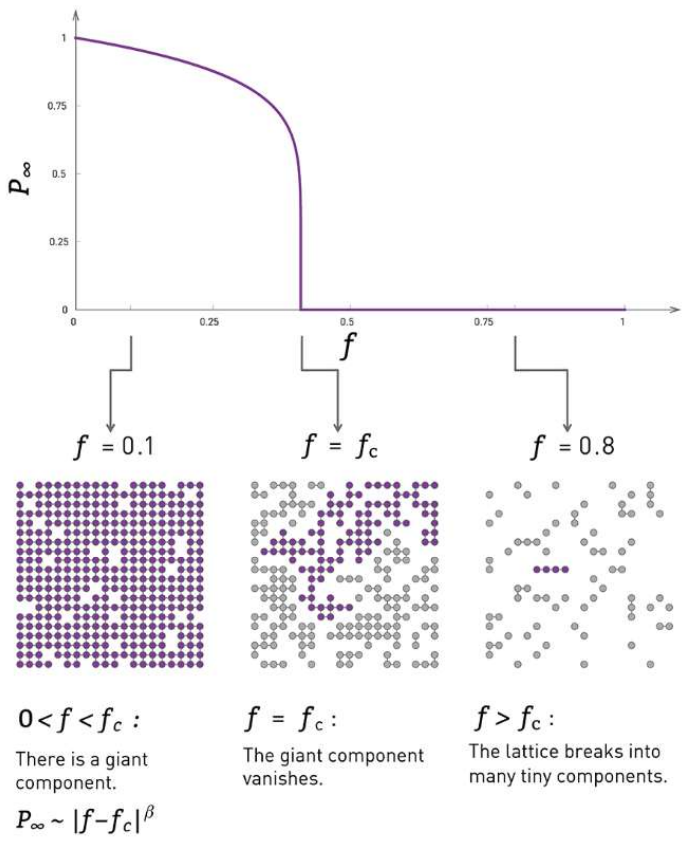


Figure 9.5: The effects of node removal are effectively described by the inverse percolation process. We start with a square lattice, viewed as a network with nodes at the intersections, and we randomly remove a fraction f of the nodes. We then measure the size of the largest component formed by the remaining nodes, represented by p_∞, the probability that a randomly chosen node is part of the largest component. The resulting network configurations are displayed in the bottom panels, with key characteristics of each phase listed below.; figure is taken from [22]

breakdown due to random node removal is abrupt rather than gradual. While removing a small fraction of nodes has minimal impact on network integrity, but reaching a critical threshold causes the network to fragment into disconnected components. This phase transition from a connected to a fragmented state can be analyzed using percolation theory in both regular and random networks. However, for scale-free networks, key aspects of this behavior differ, which we will explore in the next section.

9.6.1 Network Robustness of Scale-free Networks

Percolation theory typically addresses regular lattices, where all nodes have the same degree, or random networks, where node degrees are similar. In scale-free networks,

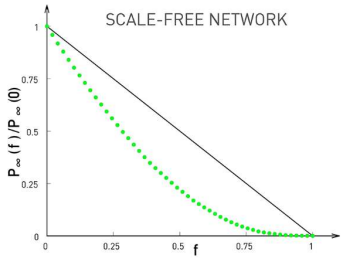


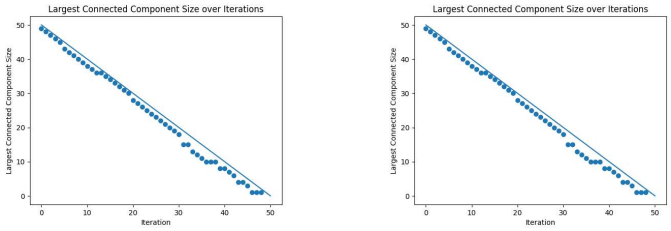
Figure 9.6: The fraction of nodes that belong to the giant component after an f fraction of nodes are removed from a scale-free network with $\gamma = 2.5$, $N = 10,000$ and $k_{min} = 1$. The figure is taken from [22].

however, hubs—nodes with significantly higher degrees—play a crucial role in the percolation transition. These hubs make the network more resilient to random node removal, as the majority of low-degree nodes can be removed without significant fragmentation. The presence of hubs alters the dynamics of the percolation process, often delaying or softening the phase transition compared to regular or random networks.

Let's study the removal of random nodes and its impact on the largest connected component in a scale-free network. In figure 9.6, we plot this behavior and we observe that the value of critical threshold is very high as compared to the random network.

Next we share the code to study the removal of nodes from BA networks u.a.r. and plot the size of the largest connected component to study its impact. In Figure 9.7, we plot the size of largest connected component as we keep removing the nodes uniformaly at randomly. We show plots for 2 random runs of the same code.

```
import random
import networkx as nx
import matplotlib.pyplot as plt

G = nx.barabasi_albert_graph(50, 3)

largest_connected_component_size=[]
for i in range(49):
  node_to_remove = random.choice(list(G.nodes()))
  #select a node to remove
  G.remove_node(node_to_remove)
  largest_connected_component_size.append(len(max(nx.connected_components(G), key=len)))

plt.scatter(range(49), largest_connected_component_size)
plt.xlabel("Iteration")
plt.ylabel("Largest Connected Component Size")
plt.title("Largest Connected Component Size over Iterations")

x = [0, 50]
y = [50, 0]
```



(a) (b)

Figure 9.7: The size of largest connected component as we remove nodes 1 by 1 from a BA network that is generated using $n = 50$, and $m = 3$. Two figures are corresponding to two different runs of the same code. Linear line shows the gradual decrease in the size of the largest connected component.

```
plt.plot(x, y, "-")
plt.show()
```

Exercise 9.1 *Create BA and Random networks of same size (having same number of nodes and same number of edges) as mentioned below.*
1. $n = 100$ *and* $m \sim 300$
2. $n = 1000$ *and* $m \sim 1000$
Plot the size of the largest connected component as the nodes are removed u.a.r. one by one until one node is left in the network in both cases. Analyze your plots and explain if you observe similar or different patterns in both cases.

A key feature of scale-free networks is their ability to endure random failures without fragmenting. This robustness stems from the presence of hubs, which are large-degree nodes. Since random node failures affect all nodes equally, but scale-free networks contain far more small-degree nodes than hubs, and the likelihood of removing a critical hub is low. Removing small nodes has minimal impact on the network's structure and functionality. For example, randomly closing a small airport has little effect on global air travel, as major hubs remain operational, maintaining connectivity.

9.6.2 Node removal vs. Edge removal

It is also interesting to study that how the network properties will behave if we remove edges instead of nodes. When links are randomly removed rather than nodes, the critical threshold f_c remains the same for both cases. This means that the network will break apart at approximately the same fraction of removed links or nodes. In Figure 9.8, we show the results for a random networks having average degree 2 that has $f_c \approx 0.5$. However, the network's giant component is smaller when nodes are removed compared to links. This is because removing a node also removes all its connected links. The removal of an f fraction of nodes is equivalent with the removal of an $f\langle k\rangle$ fraction of links, which clearly makes more damage than the removal of an f fraction of links.

Exercise 9.2 *Create a random network and plot the size of its largest connected component as we remove f fraction of nodes and edges, where f ranges from 0 to 1.*

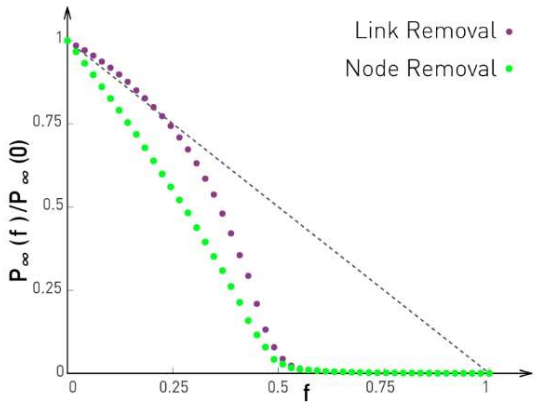


Figure 9.8: The impact of Node and Link removal on P_∞ in a random network.; figure is taken from [22]

```
import random
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np

G = nx.gnm_random_graph(1000, 2000)
G1=G.copy()

lcc_ER_node_removal=[]
for i in range(G.number_of_nodes()-1):
  node_to_remove = random.choice(list(G.nodes()))
  #select a node to remove
  G.remove_node(node_to_remove)
  lcc_ER_node_removal.append(len(max(nx.connected_components(G), key=len)))


lcc_ER_edge_removal=[]
for i in range(G1.number_of_edges()-1):
  edge_to_remove = random.choice(list(G1.edges()))
  G1.remove_edge(edge_to_remove[0], edge_to_remove[1])
  lcc_ER_edge_removal.append(len(max(nx.connected_components(G1), key=len)))


plt.plot(np.linspace(0, 1, num=len(lcc_ER_node_removal)), lcc_ER_node_removal, "r-")
plt.plot(np.linspace(0, 1, num=len(lcc_ER_edge_removal)), lcc_ER_edge_removal, "b-")
plt.legend(labels=["Node Removal", "Link Removal"])
plt.xlabel("Fc")
plt.ylabel("Largest Connected Component Size")

plt.show()
```

The results are shown in Figure 9.9.

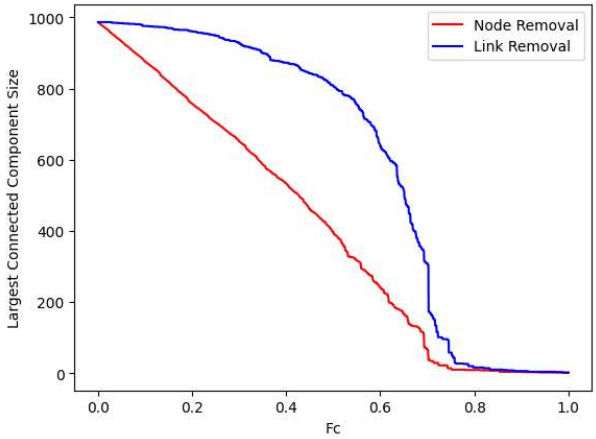


Figure 9.9: The impact of Node and Link removal on the largest connected component.; figure is taken from [22]

Exercise 9.3 *Create two BA networks of having 100 and 1000 nodes with $m = 3$. Plot the size of largest connected component as you remove fraction of nodes and edges from 0 to 1.*

In this section, we explored a key characteristic of real networks: their robustness to random failures. While random networks have a finite breakdown threshold, scale-free networks with $\gamma < 3$ exhibit a breakdown threshold that approaches one. This means nearly all nodes must be removed to dismantle a scale-free network, demonstrating their remarkable resilience to random failures.

9.6.3 Network Attacks

Now, as we know that the scale-free networks are robust to random failure of nodes, the next interesting question that comes to mind is: What if we do not remove the nodes randomly, and instead remove them strategically, e.g., removing hubs? In this scenario, we remove nodes starting with the highest degree and proceed in descending order. While this sequence of failures is unlikely under normal conditions, it simulates a targeted attack on the network. Such an approach assumes precise knowledge of the network's structure, enabling the intentional targeting of hubs to deliberately disrupt the network's functionality [16].

The removal of a single hub usually won't fragment a network, as other hubs can maintain its connectivity. However, after a few key hubs are removed, large portions of the network begin to disconnect. With continued targeting of hubs, the network quickly disintegrates into small, isolated clusters. In scale-free networks, the removal of hubs has a profound impact. While random failures do not lead to a critical threshold, targeted attacks on hubs reintroduce this threshold, and at a significantly low value. This means that removing just a small fraction of the hubs is enough to fragment the network into tiny clusters. This section aims to quantify the vulnerability of such networks to targeted attacks.

Figure 9.10 compares the resilience of a scale-free network under targeted attacks (purple) and random failures (green). In an attack scenario, nodes are removed in descending order of their degree, starting with the largest hubs, while random failures remove nodes without regard to their degree. The plot reveals the network's extreme vulnerability to attacks, as the critical threshold f_c is small, indicating that removing just a few hubs can fragment the network. The initial network parameters are: degree exponent $\gamma = 2.5$, $k_{min} = 2$, and $n = 10,000$.

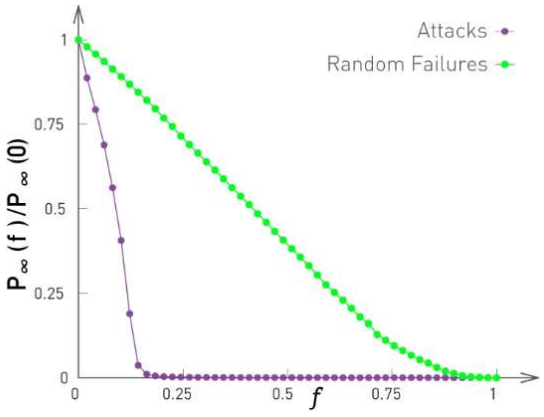


Figure 9.10: The strategic attack where we remove high degree nodes vs. random node removal on scale-free networks.; figure is taken from [22]

An attack on a scale-free network has two key consequences:

1. The critical threshold f_c is lower than $f_c = 1$, meaning that the network can be fragmented by removing only a finite fraction of its hubs.
2. The value of f_c is exceptionally small, indicating that just a tiny fraction of hub removals is enough to significantly disrupt and cripple the network.

Exercise 9.4 *Create a BA network of 1000 nodes and having 2000 edges. Apply the strategic attack where the nodes are removed based on their degree and compare the results with random removal of nodes.*

```
import random
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np

G = nx.barabasi_albert_graph(1000, 3)
G1=G.copy()

lcc_random_node_removal=[]
for i in range(G.number_of_nodes()-1):
  node_to_remove = random.choice(list(G.nodes()))
  #select a node to remove
```

```
  G.remove_node(node_to_remove)
  lcc_random_node_removal.append(len(max(nx.connected_components(G), key=len)))


lcc_highdeg_node_removal=[]
for i in range(G1.number_of_nodes()-1):
  #select highest deg node to remove
  degrees = dict(G1.degree())
  node_to_remove = max(degrees, key=degrees.get)
  G1.remove_node(node_to_remove)
  lcc_highdeg_node_removal.append(len(max(nx.connected_components(G1), key=len)))


plt.plot(np.linspace(0, 1, num=len(lcc_random_node_removal)),
                lcc_random_node_removal, "r-")
plt.plot(np.linspace(0, 1, num=len(lcc_highdeg_node_removal)),
                lcc_highdeg_node_removal, "b-")
plt.legend(labels=["Random Node Removal", "Highest Degree Node Removal"])
plt.xlabel("Fc")
plt.ylabel("Largest Connected Component Size")
plt.show()
```

The plot is shown in Figure 9.11.

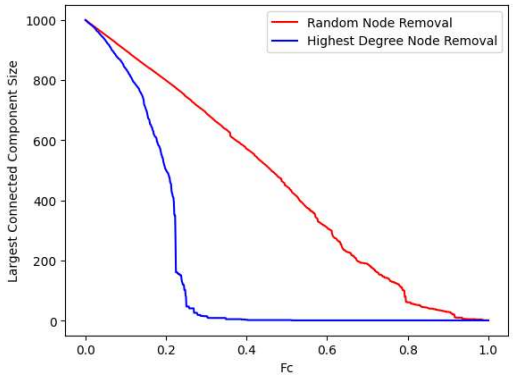


Figure 9.11: The size of largest connected component for strategic attack where we remove high degree nodes vs. random node removal on scale-free networks having $n = 1000$ and $m = 3$.

Exercise 9.5 *Create a BA network of 1000 nodes and having 2000 edges. Apply the following attack on scale-free network:*
1. Remove nodes from highest to lowest closeness centrality
2. Remove nodes from highest Pagerank to lowest PageRank
3. Remove nodes from highest core-number to lowest core-number
Plot the size of largest connected component and compare the results.

Exercise 9.6 *Create a random network having 1000 nodes and 2000 edges. Apply the following attack on scale-free network:*
1. Remove nodes u.a.r.
2. Remove nodes from highest to lowest degree
3. Remove nodes from highest to lowest closeness centrality
4. Remove nodes from highest Pagerank to lowest PageRank
5. Remove nodes from highest core-number to lowest core-number
Plot the size of largest connected component and compare the results.

9.6.4 Cascading Failures

Throughout this chapter, we assumed that node failures occur independently as random events. However, in real networks, the activity of each node is influenced by its neighbors. Therefore, the failure of one node can trigger the failure of its connected nodes [21]. Let's consider a few examples to illustrate this dependency. In Power grid failures (blackouts), after the failure of a node or link, electric currents are instantly redistributed across the remaining power grid. For instance, on August 10, 1996, in Oregon, a high-voltage line carrying 1,300 megawatts snapped when it sagged and touched a tree. The current was rerouted to two lower-voltage lines, which were not equipped to handle the increased load and subsequently failed. This caused further strain on the grid, leading to the malfunction of 13 generators and resulting in a large-scale blackout across 11 U.S. states and two Canadian provinces.

In the denial of service attack on Internet, when a router fails to transmit packets, Internet protocols notify neighboring routers to bypass the faulty equipment and reroute the packets through alternative paths. This rerouting increases traffic on other routers, which can potentially trigger a chain reaction, causing denial-of-service (DoS) attacks across the Internet. Another example is cascading in financial crisis; for instance, the 2008 decline in U.S. housing prices propagated through the financial network, triggering a chain reaction of bank, company, and even nations. This cascade ultimately led to the most severe global financial crisis since the Great Depression of the 1930s. Other famous examples are, flight delays, supply chains, and disappearance of species.

Despite coming from different domains, these examples share key characteristics. Initially, the failures had a limited impact on the network's structure. However, the failure did not remain isolated but spread through the network's connections, triggering further breakdowns. As a result, multiple nodes were unable to perform their regular functions, leading to cascading failures—a hazardous phenomenon in most networks.

The emergence of a cascading event depends on many parameters, including the structure of the network on which the cascade propagates, nature of the propagation process and the breakdown criteria of each individual component.

Failure Propagation Model: In this section, we will study that how can we model the propagation of cascading failure. Let's assume that in a given network each node contains an agent. An agent i can be in the state 0 (active or healthy) or 1 (inactive or failed), and is characterized by a breakdown threshold ϕ_i for all i. It means that a node will fail if at least a ϕ fraction of its neighbors are in state 1.

The model works as follows, At time step 0, a fixed number of nodes are failed due to the attack. After this, at every time step, we randomly select an agent and update its state following a threshold rule:

1. If the selected agent i is in state 0, it inspects the state of its neighbors. The agent i changes its state to 1 (i.e. it also fails) if at least a ϕ fraction of its neighbors are in state 1, otherwise it retains its original state 0.

2. If the selected agent i is in state 1, it does not change its state.

Figure 9.12 shows a small example where each node has same breakdown threshold $\phi = 0.4$. In practice, each node can has a different breakdown threshold based on its capacity.

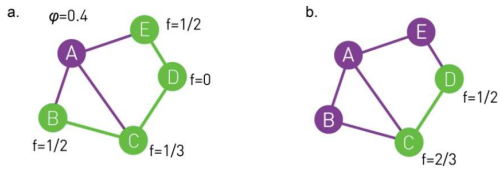


Figure 9.12: This example shows the cascade in a small network in which each node has the same breakdown threshold $\phi = 0.4$. Initially all nodes are in state 0, shown as green circles. After node A changes its state to 1 (purple), its neighbors B and E will have a fraction $f = 1/2 > 0.4$ of their neighbors in state 1. Consequently they also fail, changing their state to 1, as shown in (b). In the next time step C and D will also fail, as both have $f > 0.4$. Consequently the cascade fails the whole network, reaching a size s = 5; Figure is taken from [22].

Exercise 9.7 *Write down the code to simulate cascade failure using the above discussed method on a BA network having $n = 40$ and $m = 3$. The cascade starts from 5 highest degree nodes. Plot the network for 5 iterations of the failure where active nodes are plotted in the black color and in-active nodes are plotted in the gray color.*

```
#code of cascade failure

import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

G = nx.barabasi_albert_graph(20,2)

#breakdown threshold
phi=0.2
for each in G.nodes():
  G.nodes[each]['state'] = '0'

highdegnodes=sorted(G.degree, key=lambda x: x[1], reverse=True)[0:5]

for each in highdegnodes:
  G.nodes[each[0]]['state'] = '1'

for i in range(5):
  selected_agent = random.choice(list(G.nodes()))
```

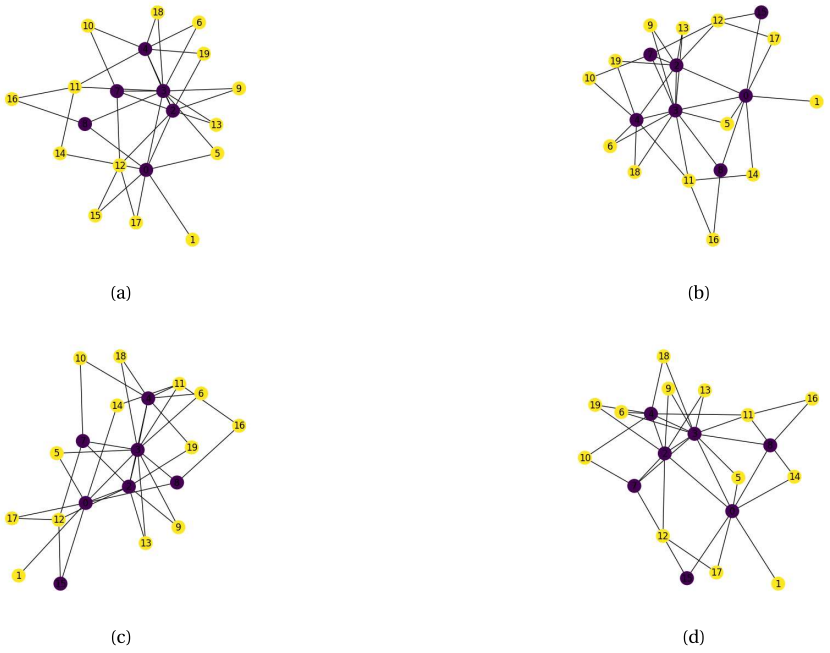


Figure 9.13: The cascade failue on a BA network with $n = 20$ and $m = 2$. The cascade starts from 5 high degree nodes and $\phi = 0.2$. At each time step a node is chosen u.a.r. and its state is updated based on the model defined above.

```
if G.nodes[selected_agent]['state']=='0':
  inactive_ngh=0
  active_ngh=0
  for ngh in G.neighbors(selected_agent):
    if G.nodes[ngh]['state']=='0':
      active_ngh+=1
    else:
      inactive_ngh+=1
  if active_ngh/(active_ngh+inactive_ngh) > phi:
    G.nodes[selected_agent]['state'] = '1'

vertex_color = [G.nodes[v]['state']=='0' for v in G.nodes()]
plt.figure(figsize=(5, 5))
nx.draw(G, node_color=vertex_color, with_labels=True)
plt.show()
```

Plots for 4 iterations are shown in Figure 9.13.

Exercise 9.8 *Write down the code to simulate cascade failure using the following model, where the cascade starts from the attacked nodes and their state is changed to inactive. At each time step:*
1. all nodes who have the fraction of failed nodes higher than ϕ changes their state to

inactive.
Apply this model on a BA network having $n = 40$ and $m = 3$. Consider two cases: (i) The cascade starts from 5 nodes chosen uniformly at randomly, and (ii) The cascade starts from 5 highest degree nodes. Take $\phi = 0.2$ and 0.4 Plot the total number of inactive nodes for 20 iterations of the failure propagation for both ϕ values.

Exercise 9.9 *Design two more models to simulate cascade failures in real-world networks. Write down their algorithms that can be implemented using any programming language.*

9.7 Branching Process

Due to the complexity of failure propagation models, analytically predicting the scaling behavior of the resulting avalanches is challenging. To determine the avalanche exponent, we employ the branching model. This is the simplest model that effectively captures the fundamental characteristics of cascading events. In this section, we will discuss branching process with the help of a contagion model.

We begin with the simplest model of contagion, which we refer to as a branching process. It works as follows:

1. First wave: Suppose that a person is infected and interacts with a population. This person transmits it to each person he meets independently with a probability of p. Further, suppose that he meets k people while he is contagious; let's call these k people the first wave of the epidemic. Based on the random transmission of the disease from the initial person, some of the people in the first wave may get infected with the disease, while others may not.

2. Second wave: Now, each person from the first wave goes out and meets k different people, resulting in a second wave of $k \cdot k = k^2$ people. Each individual infected in the first wave will independently transmit the disease to each of them with a probability p.

3. Subsequent waves.: Further waves are formed in the same way, by having each person in the current wave meet k new people, passing the disease to each independently with probability p.

The contact network for this epidemic is shown in Figure 9.14(a) (with k = 3 and only the first three waves are shown). This network is refereed as a tree. This tree has a single root node at the top, which represents the initial infected person. Every node in the tree (except the root) is connected to a node in the level above it, representing transmission of the disease. Each node is also connected to several nodes below it, symbolizing the spread to new individuals. The branching process forms an infinite tree, as waves of infection can theoretically continue indefinitely.

In this model, the behavior of an epidemic can be illustrated by tracking the edges of the contact network where successful transmissions occur. Figure 9.14(b) depicts a milder epidemic, characterized by a lower contagion probability p. In this scenario, out of the two individuals infected in the first wave, one fails to transmit the disease further, while the other infects just one additional person, who also does not pass it on. Consequently, this disease dissipates after the second wave, having infected only four people in total. In Figure 9.14(c), we see a vigorous epidemic that spreads aggressively, infecting two individuals in the first wave, three in the second, five in the third, and likely continuing to grow in subsequent waves.

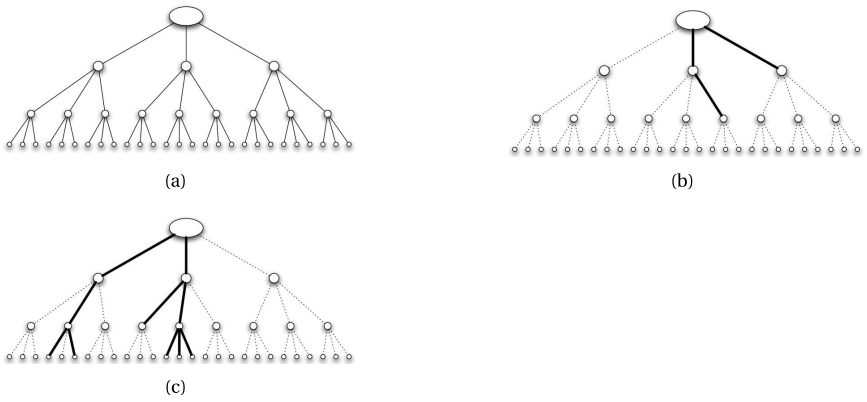


Figure 9.14: The branching process model provides a straightforward framework for analyzing the spread of an epidemic by adjusting two key factors: the amount of contact among individuals and the level of contagion. This model helps to understand how varying these parameters influences the transmission dynamics and the overall progression of the epidemic. (a) The contact network for a branching process, (b) With low contagion probability, the infection is likely to die out quickly, and (c) With high contagion probability, the infection spreads widely

9.7.1 Basic Reproductive Number

The main observation in Figure 9.14(b) highlights a key aspect of branching processes: if the disease fails to infect anyone in a particular wave, it will ultimately die out. This is because individuals in subsequent waves can only contract the disease from those higher in the tree, meaning no future infections can occur once the disease ceases to spread.

In the branching process model, there are essentially two scenarios for a disease's trajectory: it either reaches a wave where it infects no one, resulting in its extinction after a finite number of steps, or it continues to infect individuals in every wave, perpetuating its presence indefinitely within the contact network. Notably, there exists a straightforward criterion to differentiate between these two outcomes, which is determined by the **basic reproductive number** of the disease.

The basic reproductive number, denoted as R_0 represents the expected number of new cases generated by a single infected individual. In this model, since each individual meets k new people and infects each with a probability p, the basic reproductive number can be calculated as $R_0 = pk$. The outcome of the disease within the branching process model depends on the value of R_0, whether it is smaller or larger than 1

Note: If $R_0 < 1$, then with probability 1, the disease dies out after a finite number of waves. If $R_0 > 1$, then with probability greater than 0 the disease persists by infecting at least one person in each wave.

We can explain it here intuitively. When $R_0 < 1$, the disease cannot sustain itself, as each infected individual generates, on average, fewer than one new case. Consequently, even if the outbreak temporarily grows due to random fluctuations, its size will consistently decline over time. Conversely, if $R_0 > 1$, the outbreak is expected to grow, with its size trending upward. However, it is important to note that even with

$R_0 > 1$, this indicates only a positive probability of the disease persisting, not a guarantee. Since if $p < 1$, there is always a chance that the initial infected individuals fail to infect anyone, leading to the disease dying out. Thus, even a highly contagious disease can “get unlucky" and disappear from the population before it has a chance to spread significantly.

The dichotomy presented by this condition exhibits a notable "knife-edge" quality when R_0 is close to 1. For instance, if we have a branching process where R_0 is just below 1 and we slightly increase the contagion probability p, this could elevate R_0 above 1, suddenly creating a positive probability of a significant outbreak. Conversely, if we reduce the contagiousness of the disease just enough to drop R_0 below 1, we can effectively eliminate the risk of a large epidemic. Since R_0 is the product of p and k, even minor changes in the number of contacts k that each individual has can significantly impact the situation when R_0 is close to 1. This implies that near the critical value of $R_0 = 1$, significant effort can be worthwhile to achieve even minor adjustments in the basic reproductive number. Since R_0 is the product of p and k, two fundamental types of public health interventions can be understood as reductions to R_0:

1. **Quarantining individuals**: This measure decreases the contact rate k.

2. **Promoting better sanitary practices**: These behavioral changes aim to reduce the contagion probability p.

Both strategies can effectively lower R_0 and help manage the spread of the disease.

The branching process model offers a simplified perspective on disease transmission, using a very simple network, and no triangles in the network. In the upcoming sections, we will explore models that account for more intricate contact networks. In these more complex frameworks, the clear-cut dichotomy established earlier may not apply. However, the concept of the basic reproductive number remains a valuable heuristic. Even when precise conditions for epidemic persistence or extinction are unclear, R_0 serves as a useful approximation for assessing a disease's potential for spread.

9.8 Epidemic Models

In this section, we will discuss epidemic models that has always been of great interest for Epidemiologists and network scientists.

9.8.1 The deterministic SIR model

To model the spread of an epidemic by direct contact in a network, we can use a compartmental model, where each node (representing an individual) can be in one of three possible stages:

1. **Susceptible (S)**: An individual who is healthy but can contract the disease if they come into contact with an infected individual.

2. **Infected (I)**: An individual who has contracted the disease and can spread it to susceptible individuals.

3. **Recovered (R)**: An individual who has recovered from the disease and is now immune, meaning they cannot be infected again or spread the disease.

This type of model is known as the SIR model. The transitions between these stages are typically governed by the following processes:

- **Infection:** A susceptible individual becomes infected after coming into contact with an infected individual. The rate at which this happens is often denoted by β, which represents the transmission rate.
- **Recovery:** An infected individual eventually recovers from the disease and moves to the recovered stage. The rate at which this happens is denoted by γ, which represents the recovery rate.

The SIR Model Dynamics

In a simple SIR model, the population is assumed to be closed (no births or deaths during the period of the epidemic), and the model is typically described by a set of differential equations. If N be the total size of the population, the S + I + R = N. Under the assumption of homogeneous mixing, i.e. anyone is likely to infect anyone else, we have the following rate equations:

1. Susceptible individuals:
   $\frac{dS}{dt} = -\beta SI$
   This equation represents the rate at which susceptible individuals become infected.
2. Infected individuals:
   $\frac{dI}{dt} = \beta SI - \gamma I$
   This equation represents the change in the number of infected individuals, which increases as susceptibles become infected and decreases as infected individuals recover.
3. Recovered individuals:
   $\frac{dR}{dt} = \gamma I$
   This equation represents the rate at which infected individuals recover.

Assumptions

- Homogeneous Mixing: Every individual in the population has an equal chance of coming into contact with every other individual.
- Fixed Population: The total population remains constant, with $S(t) + I(t) + R(t) = N$ where N is the total population size.
- No Re-infection: Once recovered, individuals are assumed to gain immunity and do not become susceptible again.

Epidemic Spreading on Small-World Networks

In reality, social relations and physical/geographical proximity affect the probability to get infected and make some people more likely to be infected than others. To model this, we need to go beyond the assumptions of homogeneous mixing. To

model the spread of an epidemic more realistically, we can incorporate the role of contact structure (spreading through a network) and role of space (diffusion of an epidemic front) into the SIR dynamics.

Implementation

Here, we discuss that how can we simulate SIR Model using NetworkX in Python. Let's assume that we have the population network G where all nodes are susceptible (state = 's') at the start. Initially we have a set of infected nodes and their state is changed to 'i'. Now at each iteration, each infected node will try to infect each of its neighbor with probability β and it will change its state to recovered with probability γ. The propagation will stop when there is no infected node in the network.

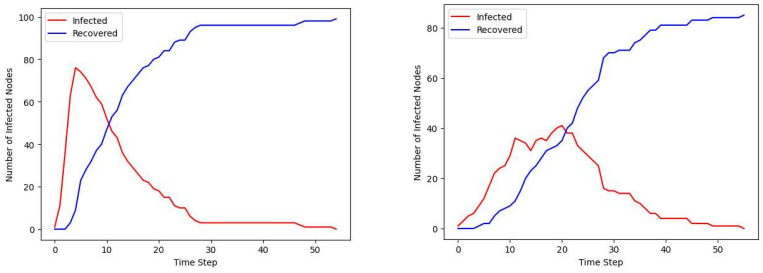
The following code shows the simulation of SIR model with.

```
def simulate_sir(graph, initial_infected_nodes, infection_probability,
                    recovery_probability):
  """
  Simulates the SIR spreading model on a given graph.
  Input:
    graph: A NetworkX graph.
    initial_infected_nodes: A list of node IDs that are initially infected.
    infection_probability: The probability of infecting a susceptible node (beta).
    recovery_probability: The probability of recovery for an infected node (gamma).

  Returns:
    Lists containing the total number of infected and recovered nodes
                    at each time step.
  """

  # Initialize node states (S: Susceptible, I: Infected, R: Recovered)
  for node in graph.nodes():
    graph.nodes[node]['state'] = 'S'
  for node in initial_infected_nodes:
    graph.nodes[node]['state'] = 'I'

  #the following compute the total number of infected nodes at each iteartion
  infected_counts = []
  infected_nodes_count = sum(1 for node in graph.nodes()
                        if graph.nodes[node]['state'] == 'I')
  infected_counts.append(infected_nodes_count)

  recovered_counts = []
  recovered_nodes_count = sum(1 for node in graph.nodes()
                        if graph.nodes[node]['state'] == 'R')
  recovered_counts.append(recovered_nodes_count)

  while True:
    newly_infected_nodes = []
    newly_recovered_nodes = []
```

```
    for node in graph.nodes():
      if graph.nodes[node]['state'] == 'I':
        # Try to infect susceptible neighbors
        for neighbor in graph.neighbors(node):
          if graph.nodes[neighbor]['state'] == 'S' and
                    random.random() < infection_probability:
            newly_infected_nodes.append(neighbor)
        # Try to recover
        if random.random() < recovery_probability:
          newly_recovered_nodes.append(node)

    # Update node states
    for node in newly_infected_nodes:
      graph.nodes[node]['state'] = 'I'
    for node in newly_recovered_nodes:
      graph.nodes[node]['state'] = 'R'

    #code to retun number of infected nodes at each time step
    infected_nodes_count = sum(1 for node in graph.nodes()
                if graph.nodes[node]['state'] == 'I')
    infected_counts.append(infected_nodes_count)
    recovered_nodes_count = sum(1 for node in graph.nodes()
                if graph.nodes[node]['state'] == 'R')
    recovered_counts.append(recovered_nodes_count)

    # Check if the epidemic has ended
    if all(graph.nodes[node]['state'] != 'I' for node in graph.nodes()):
      break

  return infected_counts, recovered_counts

#code to execute SIR model on BA network
G = nx.barabasi_albert_graph(100,2)  # Create a random graph
initial_infected_nodes = [1]
infection_probability = 0.1
recovery_probability = 0.1

infected_counts_time_step,recovered_counts_time_step = simulate_sir(G,
            initial_infected_nodes, infection_probability, recovery_probability)
plt.plot(infected_counts_time_step , "r")
plt.plot(recovered_counts_time_step, "b")
plt.legend(labels=["Infected", "Recovered"])
plt.xlabel("Time Step")
plt.ylabel("Number of Infected Nodes")
plt.show()
```

In Figure 9.15, we show the total number of infected and recovered nodes when we run SIR model on a BA network having $n = 100$ and $m = 2$ for the foloowing two cases: (i) $\beta = 0.5$ and $\gamma = 0.1$, and (ii) $\beta = 0.1$ and $\gamma = 0.1$

As you observed that SIR model is a stochastic model and the total number of



(a) (b)

Figure 9.15: Total number of infected and recovered nodes on a BA network (n=100 and m=2) for (i) $\beta = 0.5$ and $\gamma = 0.1$, and (ii) $\beta = 0.1$ and $\gamma = 0.1$

infected nodes might differ for different run of the code. In practice, we execute the model until it converge to get the expected number of infected and recovered nodes at each iteration. A simple solution for this is to repeat the model k number of times and take the average of all values.

Exercise 9.10 *Write the code to run the SIR model 10 times and compute the average and standard deviation for the total number of infected and recovered nodes at each time step.*

Exercise 9.11 *Create a BA network for $n = 100$ and $m = 2$. Now, plot the final total number of infected and recovered nodes (count these nodes when the SIR model stops and no node remains infected) as we change the number of initially infected nodes as follows: 2, 4, 6, 8, 10.*

9.8.2 Variation of SIR Models

SI Model

In SI model, each individual can either be infected or susceptible. It works as follows:

- Initially, some nodes are in the I state and all others are in the S state.
- Each node that becomes infected and is in I state remains infectious thereafter.
- At each time step, infected nodes will pass the infection to its susceptible neighbors with probability p.

SIS Model

In SIS model, individuals become susceptible again from infected state, meaning no immunity is gained. The mechanics of the model closely follow the SIR process. It works as follows:

- Initially, some nodes are in the I state and all others are in the S state.

- Each node that becomes infected and is in I state remains infectious for a fixed number of time steps t_I.
- During each of these t_I steps, infected node v will pass the infection to its susceptible neighbors with probability α.
- After t_I steps, node v is no longer infectious, and it returns to the S state.

Exercise 9.12 *1. Write the code to simulate the SIS model on a network with the given infection probabilities α, and t_I.*
2. Create a random network having 1,000 nodes and 5,000 edges. Now choose two highest degree nodes as infected node at the initial state and execute the SIS model. Plot the number of infected nodes for the starting 10 iterations for the following two scenarios:
a. $\alpha = 0.2$, and $t_I = 5$
b. $\alpha = 0.4$, and $t_I = 10$ 3. Interpret your results. For better understanding try other combinations of α and t_I.

Exercise 9.13 *1. Write the code to simulate an another version of SIS model on a network with the given infection probabilities α, i.e., probability to be infected from susceptible, and β, i.e., the probability to be susceptible from infected state.*
2. Create a BA network having 1,000 nodes and 5,000 edges. Now choose two highest degree nodes as infected nodes at the initial state and execute the SIS model. Plot the number of infected nodes for the starting 10 iterations for the following two scenarios:
a. $\alpha = 0.2$, and $\beta = 0.2$
b. $\alpha = 0.2$, and $\beta = 0.1$ 3. Interpret your results. For better understanding try other combinations of α and β.

SIRS Model

In many diseases, once you are recovered, after some time you can again be susceptible to this disease and such epidemics can be modeled using SIRS model. In SIRS model, initially, some nodes are in the I state and all others are in the S state. Each node v that enters the I state remains infectious for a fixed number of steps t_I. During each of these t_I steps, v has a probability α of passing the disease to each of its susceptible neighbors. After t_I steps, node v is no longer infectious. It then enters the R state for a fixed number of steps t_R. During this time, it cannot be infected with the disease, nor does it transmit the disease to other nodes. After t_R steps in the R state, node v returns to the S state.

SEIR Model

SEIR Model introduces an Exposed (E) stage for individuals who have been infected but are not yet infectious.

Exercise 9.14 *Write an algorithm to simulate SIER model on a given network, where the set of initially infected node, and the probability of following state change: S-E, E-I, I-R, are given.*

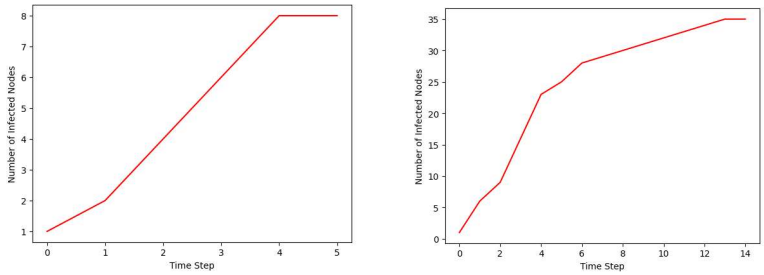
9.9 Independent Cascade Model

In Independent Cascade Model (ICM) [17], the infection is started from a source node or a group of source nodes. When a node is infected, in the next iteration, it tries to infect all of its neighbors with the given influence probabilities, and it will not infect any of its neighbors in further iterations. Once there is no newly infected node in an iteration, the propagation process is stopped. The total number of infected nodes denotes the spreading power or influencing power of the seed nodes.

In the ICM model, a node's influential power is determined as follows. The propagation is started from the given node, and the process is repeated several times. The average of the number of infected nodes in all iterations is considered the spreading or influential power of the seed node.

Next we implement ICM model where the list of initially infected nodes and infection probability is given.

```
import networkx as nx
import random
import matplotlib.pyplot as plt

def simulate_independent_cascade(G, initial_infected_nodes, infection_prob):
    """
    Simulates the Independent Cascade model on a given graph.

    Input:
        G: A NetworkX graph.
        initial_infected_nodes: A list of node IDs that are initially infected.
        infection_prob: The probability of infection when a susceptible
        node is in contact with an infected node.

    Returns:
        A list containing the total number of infected nodes at each time step.
    """

    # Initialize node states (0: Susceptible, 1: Infected)
    for node in G.nodes():
        G.nodes[node]['state'] = 0
    for node in initial_infected_nodes:
        G.nodes[node]['state'] = 1

    infected_counts = []
    infected_nodes_count = sum(1 for node in G.nodes()
                               if G.nodes[node]['state'] == 1)
    infected_counts.append(infected_nodes_count)

    newly_infected_nodes = set(initial_infected_nodes)

    while newly_infected_nodes:
        newly_infected_in_this_round = set()
        for infected_node in newly_infected_nodes:
            for neighbor in G.neighbors(infected_node):
```



(a) (b)

Figure 9.16: Total number of infected and recovered nodes on a BA network (n=100 and m=2) for (i) infection probability = 0.1 and (ii) infection probability = 0.2

```
                if G.nodes[neighbor]['state'] == 0:
                    if random.random() < infection_prob:
                        G.nodes[neighbor]['state'] = 1
                        newly_infected_in_this_round.add(neighbor)

        newly_infected_nodes = newly_infected_in_this_round
        infected_nodes_count = sum(1 for node in G.nodes()
                 if G.nodes[node]['state'] == 1)
        infected_counts.append(infected_nodes_count)
    return infected_counts

G = nx.barabasi_albert_graph(100, 2)  # Create a random graph
initial_infected_nodes = [1]
infection_prob = 0.2

infected_counts_time_step = simulate_independent_cascade(
    G, initial_infected_nodes, infection_prob)

plt.plot(infected_counts_time_step, "r")
plt.xlabel("Time Step")
plt.ylabel("Number of Infected Nodes")
plt.show()
```

We simulate ICM model on a BA network $n = 100$ and $m = 2$ for the following two cases: (i) infection probability = 0.1 and (ii) infection probability = 0.2. The infection is started from node 1 and the results are shown in Figure 9.16.

ICM is also a stochastic model so you need to run it until it converge to compute the expected number of infected nodes at each time step. In this model, we assumed that the infection probability of each edge is the same; however, in practice, each edge can also has a different influence probability.

Exercise 9.15 *Create a BA network having $n = 1000$ and $m = 3$. Plot the expected number of total infected nodes when the number of initially infected nodes varies as 2, 4, 6, 8, 10 and the initially infected nodes are the highest degree nodes. If there are more nodes having the same degree than you need to choose then break the tie by u.a.r.*

selecting the nodes. To compute the expected number of infected nodes, run the ICM model 100 times and compute the average.

9.9.1 Competitive Influence Propagation

The ICM model has been extended to model if two competitive information propagates in the network, and it is called the Competitive Independent Cascade Model (CICM) [18]. In CICM, there are two group of seed nodes and infection probability for each information, respectively.

CICM model is used in marketing, where two or more organizations want to select seed nodes to spread the information so that they can reach to the highest population size. CICM is also used for spreading awareness or correct information if some misinformation is propagating in the network.

Exercise 9.16 *1. Write Python code to simulate CICM model on a given network where the set of initially infected nodes and the propagation probability for both information is given.*
2. Create a random network having $n = 10000$ *and* $m = 30000$*. Simulate the CICM model on this network where the two information propagates as follows:*
a. First information has 5 highest degree nodes as initially infected nodes and the propagation probability is 0.2.
b. Second information has 5 randomly chosen nodes (excluding 5 highest degree nodes chosen earlier) as initially infected nodes and the propagation probability is 0.2.
Plot the total number of infected nodes for each iteration and analyze your results.

9.10 Linear Threshold Model

In real-world, it is observed that a user accepts or agrees with an information if the number of its friends believing in that information is higher than a particular threshold value. The threshold might be different for different users. In the Linear Threshold Model (LTM) [19], the information spreading starts from a group of source nodes. In each iteration, the new nodes will be influenced/infected if the fraction of infected neighbors is more than their threshold value. The infection process will stop if no new node is influenced in an iteration. This model is also known as Tipping Model when the threshold value for each node is the same [20].

Next, we provide the code to simulate LTM on a given network, list of initially infected node and the threshold value.

```
def simulate_linear_threshold(G, initial_infected_nodes, threshold):
    """
    Simulates the Linear Threshold model on a given graph.
    Input:
        G: A NetworkX graph.
        initial_infected_nodes: A list of node IDs that are initially infected.
        threshold: The infection threshold for each node.

    Returns:
        A list containing the total number of infected nodes at each time step.
    """
```

```
    # Initialize node states (0: Susceptible, 1: Infected)
    for node in G.nodes():
        G.nodes[node]['state'] = 0
    for node in initial_infected_nodes:
        G.nodes[node]['state'] = 1

    infected_counts = []
    infected_nodes_count = sum(1 for node in G.nodes() if G.nodes[node]['state'] == 1)
    infected_counts.append(infected_nodes_count)
    newly_infected_nodes = set(initial_infected_nodes)

    while newly_infected_nodes:
        newly_infected_in_this_round = set()
        for node in G.nodes():
            if G.nodes[node]['state'] == 0:
                infected_neighbors_count = 0
                total_neighbors_count = 0
                for neighbor in G.neighbors(node):
                    if G.nodes[neighbor]['state'] == 1:
                        infected_neighbors_count += 1
                    total_neighbors_count += 1

                if total_neighbors_count > 0 and (
                        infected_neighbors_count / total_neighbors_count) >= threshold:
                    G.nodes[node]['state'] = 1
                    newly_infected_in_this_round.add(node)

        newly_infected_nodes = newly_infected_in_this_round
        infected_nodes_count = sum(
            1 for node in G.nodes() if G.nodes[node]['state'] == 1)
        infected_counts.append(infected_nodes_count)

    return infected_counts

# Example Usage
G = nx.barabasi_albert_graph(100, 2)  # Create a random graph
initial_infected_nodes = [1 , 2]
threshold = 0.5

infected_counts_time_step = simulate_linear_threshold(
    G, initial_infected_nodes, threshold)

plt.plot(infected_counts_time_step, "r")
plt.xlabel("Time Step")
plt.ylabel("Number of Infected Nodes")
plt.show()
```

In Figure 9.17, we show the total number of infected nodes on a BA network $n = 100$ and $m = 2$, where the infection starts from 5 highest degree nodes and the threshold value is 0.5.

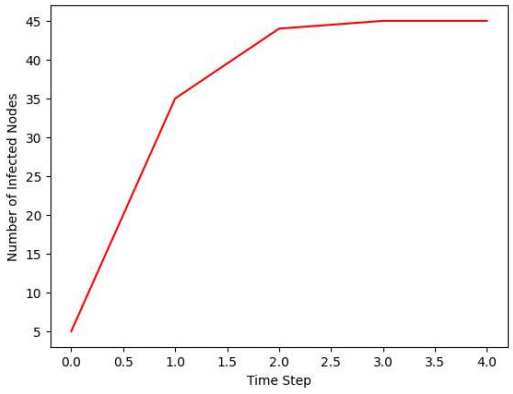


Figure 9.17: LTM model simulation on a BA network.

Exercise 9.17 *Compare the results of LTM model when the infection starts from the highest degree nodes versus u.a.r. chosen nodes; other parameters remain constants, such as the number of initially infected nodes, threshold value and the network.*

LTM model mainly simulates how people form opinion in a network. LTM model is deterministic and every run will return you the same results.

9.11 Network Immunization

Immunization strategies focus on distributing vaccines, treatments, or drugs within a population in a way that maximizes their impact. Ideally, all infected individuals or those at risk of contracting the pathogen should receive the treatment. However, challenges such as cost, difficulty in reaching at-risk individuals, and concerns about side effects often prevent full coverage. In light of these limitations, immunization strategies are designed to minimize the risk of a pandemic by distributing the available resources as efficiently as possible. Immunization is also used to immunize nodes on social media so that they are unaffected or resistant to the incoming information and will not share it further.

The primary goal of immunization is to protect the immunized individual from infection. Equally important, though, is its secondary effect: immunization slows down the spread of the pathogen within the population. For instance, when a randomly selected fraction g of individuals is immunized, the reduced number of susceptible individuals hinders the transmission of the disease, decreasing its spread rate across the population.

In a network there are two main strategies that we use to immunize the network: (i) immunize nodes and (ii) immunize edges.

9.11.1 Immunize Nodes

In Immunizing nodes, when a node is selected to immunize, it becomes inactive and will not influence/infect any of its neighbor. In simple words, we can consider that this node has been removed from the network with all of its edges. In real-world

networks, where we know the network structure and the initially infected nodes, our goal is to find a set of nodes whose immunization will minimize the spread over the network. The number of immunized nodes depend on the available budget. In a simple case, we can assume that the cost of immunizing each node is same, though in complex cases, the cost of immunizing different nodes can be different. For example, if you want a person to not share an information further then high degree (popular) nodes may ask for more price than others. Similarly, in case of pandemic, the cost of immunizing a person might be higher given the health conditions than others.

In scale-free networks, hubs play a very critical roles in spreading the influence, and therefore, a strategic immunization method should use this information.

Next, we provide code to immunize nodes u.a.r. and based on highest degree in a given network. We execute this code on a BA network having $n = 1000$ and $m = 3$, and an ER network having $n = 1000$ and $m = 3000$. We plot the total number of infected nodes in both cases when the infection was started from 5 u.a.r. chosen nodes using SIR model. The plot is shown in Figure 9.18.

```
n = 1000
m = 3
infection_probability = 0.1
recovery_probability = 0.1
num_immunized_nodes = 100  # Number of nodes to immunize
# Create BA network
ba_graph = nx.barabasi_albert_graph(n, m)
# Immunize nodes randomly
ba_graph_random_immunized = ba_graph.copy()
nodes_to_immunize = random.sample(list(ba_graph_random_immunized.nodes()),
                num_immunized_nodes)
for node in nodes_to_immunize:
  ba_graph_random_immunized.remove_node(node)

# Immunize nodes based on highest degree
ba_graph_degree_immunized = ba_graph.copy()
degrees = dict(ba_graph_degree_immunized.degree())
nodes_sorted_by_degree = sorted(degrees, key=degrees.get, reverse=True)
nodes_to_immunize = nodes_sorted_by_degree[:num_immunized_nodes]
for node in nodes_to_immunize:
  ba_graph_degree_immunized.remove_node(node)

# Start infection from 5 random nodes
initial_infected_nodes = random.sample(list(ba_graph_random_immunized.nodes()), 5)

# Simulate SIR on BA network with random immunization
infected_counts_ba_random = simulate_sir(ba_graph_random_immunized,
        initial_infected_nodes, infection_probability, recovery_probability)

# Simulate SIR on BA network with degree-based immunization
initial_infected_nodes = random.sample(list(ba_graph_degree_immunized.nodes()), 5)
infected_counts_ba_degree = simulate_sir(ba_graph_degree_immunized,
        initial_infected_nodes, infection_probability, recovery_probability)
```

```
# Create ER network
er_graph = nx.gnm_random_graph(n, 3000)
# Immunize nodes randomly
er_graph_random_immunized = er_graph.copy()
nodes_to_immunize = random.sample(list(er_graph_random_immunized.nodes()),
                num_immunized_nodes)
for node in nodes_to_immunize:
  er_graph_random_immunized.remove_node(node)

# Immunize nodes based on highest degree
er_graph_degree_immunized = er_graph.copy()
degrees = dict(er_graph_degree_immunized.degree())
nodes_sorted_by_degree = sorted(degrees, key=degrees.get, reverse=True)
nodes_to_immunize = nodes_sorted_by_degree[:num_immunized_nodes]
for node in nodes_to_immunize:
  er_graph_degree_immunized.remove_node(node)

# Start infection from 5 random nodes
initial_infected_nodes = random.sample(list(er_graph_random_immunized.nodes()), 5)

# Simulate SIR on ER network with random immunization
infected_counts_er_random = simulate_sir(er_graph_random_immunized,
        initial_infected_nodes, infection_probability, recovery_probability)

# Simulate SIR on ER network with degree-based immunization
initial_infected_nodes = random.sample(list(er_graph_degree_immunized.nodes()), 5)
infected_counts_er_degree = simulate_sir(er_graph_degree_immunized,
        initial_infected_nodes, infection_probability, recovery_probability)

# Plot the results
plt.figure(figsize=(10, 5))
plt.plot(infected_counts_ba_random, label='BA - Random Immunization')
plt.plot(infected_counts_ba_degree, label='BA - Degree-Based Immunization')
plt.plot(infected_counts_er_random, label='ER - Random Immunization')
plt.plot(infected_counts_er_degree, label='ER - Degree-Based Immunization')
plt.xlabel('Time Step')
plt.ylabel('Number of Infected Nodes')
plt.title('SIR Simulation with Immunization Strategies')
plt.legend()
plt.show()
```

9.11.2 Immunize Edges

Another way to control the propagation is by immunizing edges. In case of epidemic, it replicates the cases where each individual is asked to meet only a limited number of people or asked not to meet some specific people, such as elderly people who are more susceptible. In case of information propagation, this is even more interesting, where immunizing an edge means that the information posted by one of

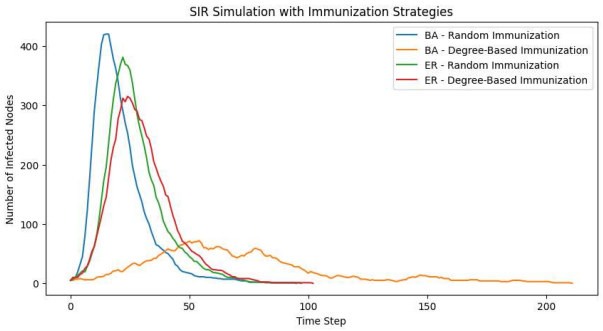


Figure 9.18: Node immunization by choosing nodes u.a.r. and based on highest degree. To simulate we have BA and ER networks having 1000 nodes and 3000 edges. In SIR model infection probability and recovery probability are set to 0.1. We immunize 100 nodes for each case and plot the total number of infected nodes at each time step.

these users will not be visible to other. This kind of process can be implemented by the platform managers to control the propagation of a given information.

Exercise 9.18 *Simulate the edge immunization method on a given network and given influence propagation model. Use the following two cases for edge immunization:*
1. immunize edges u.a.r
2. immunize edges with highest end degree product. For an edge (u, v), the end deg product is computed as $deg(u) * deg(v)$
Create a plot for edge immunization as we have shown for node immunization in Figure 9.18.

Homework

Homework 9.1 ***Simulating Network Attacks (40%)***

Consider an undirected, unweighted network $G = (V, E)$ *in which all nodes are initially active. An attacker aims to disrupt network connectivity by progressively deactivating nodes. The attack strategy operates as follows:*

1. ***Initialization (time step*** $t = 0$***):*** *The attacker receives a list of nodes,* `initial_attack_list`*, and deactivates all nodes in this list. All other nodes remain active.*

2. ***Attack (time step*** $t \geq 1$***):***

   (a) *Identify all active nodes having at least one inactive neighbor.*

   (b) *From these candidate inactive nodes, select the node having the highest degree in the original network G. If multiple candidates have the same highest degree, select one uniformly at random.*

   (c) *Deactivate the selected node.*

   Exactly one node is deactivated per time step.

3. ***Termination:*** *Stop when no eligible active node remains or the specified maximum number of time steps has been reached.*

At each time step, measure the size of the largest connected component (LCC) of the subgraph induced by the active nodes. Inactive nodes must be excluded from this calculation. If no active nodes remain, the LCC size is zero.

Tasks:

(a) (20%) Implement the attack strategy as a function that takes the following inputs:

- *Network G;*
- *`initial_attack_list`;*
- *Maximum number of attack iterations T.*

The function should return the LCC size at each time step, including $t = 0$ (after the initial deactivation).

(b) (10%) Generate a Barabási–Albert (BA) network with $n = 1000$ nodes and attachment parameter $m = 3$.

Compare the following two initial attack strategies:

(a) Random initialization: select 5 distinct nodes uniformly at random.

(b) Degree-based initialization: select the 5 nodes having the highest degrees in G.

Run each attack strategy for $T = 20$ iterations on the same generated network.

Plot the LCC size against time steps $t = 0, 1, \ldots, 20$ for both strategies in a single figure.

If an attack terminates before 20 iterations, keep the final LCC size constant for the remaining time steps.

(c) (5%) Repeat part (b) using an Erdős–Rényi (ER) random network $G(n, M)$ with exactly $n = 1000$ nodes and $M = 3000$ edges.

Compare the following initial attack strategies:

(a) 3 distinct nodes selected uniformly at random.

(b) The 3 highest-degree nodes in G.

Plot the LCC size for $t = 0, 1, \ldots, 20$ for both strategies in one figure.

(d) (5%) Briefly compare the observations from parts (b) and (c). Discuss:

- *The impact of random versus degree-based initial attacks.*
- *Differences in robustness between BA and ER networks.*
- *How network topology and the initial attacked nodes affect the subsequent attack propagation.*

Homework **9.2** ***Controlling Misinformation through Network Immunization (25%)***

A network administrator learns that a group of nodes will initiate misinformation propagation. The administrator receives a list of these nodes, denoted by `initial_propagator`*, and aims to limit the spread by immunizing selected nodes before propagation begins.*

The immunization strategy is defined as follows:

1. ***Identify candidate nodes:*** *Construct a set N containing all nodes whose shortest-path distance from at least one initial propagator is 1 or 2.*

   Exclude the initial propagators themselves from N. Each candidate node must appear only once in N.

2. ***Select nodes for immunization:*** *Select the k nodes in N having the highest degrees in the original network G.*

   If multiple nodes have equal degrees, break ties uniformly at random. If $|N| < k$, immunize all nodes in N.

An immunized node remains in the network but cannot become influenced or propagate misinformation.

Tasks:

(a) (15%) Implement the proposed immunization strategy as a function taking the following inputs:

- *Network G;*
- `initial_propagator`*: list of initial misinformation propagators;*
- *k: maximum number of nodes to immunize.*

The function should return the set of immunized nodes.

(b) (10%) Generate a BA network with $n = 1000$ and $m = 3$. Select 10 distinct initial propagators uniformly at random.

Simulate misinformation propagation using the Independent Cascade (IC) model, with transmission probability $p = 0.4$ for every edge.

Compare the following two scenarios:

(a) ***Without immunization:*** *Run the IC model on the original network.*

(b) ***With immunization:*** *Apply the proposed immunization strategy with $k = 10$ before running the IC model.*

In both scenarios, use the same network and the same initial propagators.

Use the standard IC model:

- *At $t = 0$, only the initial propagators are active.*
- *At each subsequent time step, nodes newly influenced in the previous step attempt to influence each susceptible, non-immunized neighbor independently with probability $p = 0.4$.*
- *Each influenced node has only one opportunity to influence each eligible neighbor.*
- *Once influenced, a node remains influenced.*

- *Propagation terminates when no new nodes become influenced.*

Plot the cumulative number of influenced nodes against time steps $t = 0, 1, \ldots, 10$ for both scenarios in a single figure.

If propagation terminates before time step 10, keep the cumulative number of influenced nodes remain constant for the rest of the time steps.

Include the plot in your report and briefly explain the effect of immunization on misinformation propagation.

Important points to note:

- *Immunization takes place before misinformation propagation begins.*
- *Initial propagators cannot be immunized.*
- *The number of influenced nodes is cumulative, including the initial propagators.*
- *Because IC propagation is stochastic, repeat both scenarios for at least 30 independent simulation runs and plot the average cumulative number of influenced nodes at each time step.*

Homework 9.3 ***Competitive Linear Threshold Model (35%)***

Consider an undirected, unweighted network $G = (V, E)$ in which two different types of information compete to influence nodes.

Each node can be in exactly one of three states:

- *0: uninfluenced;*
- *1: influenced by information 1;*
- *2: influenced by information 2.*

The Competitive Linear Threshold Model (CLTM) uses two threshold values, θ_1 and θ_2, which control the adoption of information 1 and information 2, respectively.

At each time step, for every uninfluenced node v, calculate

$$f_1(v) = \frac{|N(v) \cap I_1|}{|N(v)|}, \qquad f_2(v) = \frac{|N(v) \cap I_2|}{|N(v)|},$$

where $N(v)$ is the set of neighbors of v, and I_1 and I_2 are the sets of nodes currently influenced by information 1 and information 2, respectively.

For isolated nodes (degree zero), define both fractions as zero.

The update rules are:

1. *If $f_1(v) \geq \theta_1$, node v adopts information 1.*
2. *Otherwise, if $f_2(v) \geq \theta_2$, node v adopts information 2.*
3. *Otherwise, node v remains uninfluenced.*

Important conditions:

- *If both thresholds are satisfied simultaneously, information 1 has priority.*
- *Once a node adopts either information, its state never changes.*

- *All nodes must be updated simultaneously. Decisions at time $t+1$ must use node states from time t.*
- *Propagation stops when no additional nodes adopt either information.*

Tasks:

(a) (20%) Implement the Competitive Linear Threshold Model as a function with the following five inputs:

- *Network G;*
- `initial_infected_nodes_1`;
- `initial_infected_nodes_2`;
- `threshold_1`;
- `threshold_2`.

The two initial seed sets must be disjoint.

The function should simulate propagation until convergence and return the cumulative number of nodes influenced by each information type at every time step, including time step $t = 0$.

(b) (15%) Generate a BA network with $n = 1000$ and $m = 3$.

Select initial seed nodes as follows:

***Seed set 1:** Select 5 distinct nodes forming a connected subgraph using the following greedy high-degree selection method:*

(a) Select one node having the highest degree in G. Break ties uniformly at random.
(b) At each subsequent step, identify unselected nodes that are neighbors of at least one previously selected node.
(c) From these candidates, select the node with the highest degree in G. Break ties uniformly at random.
(d) Repeat until 5 nodes have been selected.

These 5 nodes form `initial_infected_nodes_1`.

***Seed set 2:** Select the 5 highest-degree nodes from $V \setminus$* `initial_infected_nodes_1`.

Break degree ties uniformly at random. These nodes form `initial_infected_nodes_2`.

Use the following thresholds:

$$\theta_1 = 0.2, \qquad \theta_2 = 0.4.$$

Run the CLTM until no additional nodes become influenced.

Create one figure showing:

- *The cumulative number of nodes influenced by information 1 and information 2 at each time step. Please note that both information types start propagating at time step $t = 0$. All non-seed nodes are initially uninfluenced.*

Use different colors and a legend to distinguish the two information types. Include the plot in your report and briefly discuss which information type spreads more widely and how the initial seed selection, threshold values, and priority rule influence the results.

9.12 Conclusions

Some important conclusions from the above discussion are

Take home messages:

- Dynamical processes on graph feature dynamically changing states of networks, that are related to states of nodes.
- Studied Percolation Theory and how percolation differs on scale-free networks than random network. In scale-free networks, hubs play a critical role in the robustness of the network.
- Studied Strategic attack by removing nodes and edges in the network.
- In epidemiological models the spread of viruses, diseases, or information in networks is investigated and one distinguishes SI, SIS, and SIR models.
- Deterministic simulations with discrete state can be implemented, e.g. in Python, using transition functions.
- Discussed Independent Cascade model, Competitive Independent Cascade model, and Linear Threshold model.
- Network immunization using node or edge immunization in a network and their simulation.

Bibliography

[1] Brodal, Gerth S. (1996), "Worst-Case Efficient Priority Queues", Proc. 7th Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 52–58

[2] Gillespie, Daniel T. Markov processes: an introduction for physical scientists. Elsevier, 1991.

[3] http://openflights.org/data.html

[4] Ognyanova, Katherine: Network Analysis and Visualization with R and igraph, www.kateto.net, NetSciX 2016 School of Code Workshop, Wroclaw, Poland

[5] Castellano, Claudio, and Romualdo Pastor-Satorras. "Non-mean-field behavior of the contact process on scale-free networks." Physical review letters 96.3 (2006): 038701.

[6] Feres, Renato. "Notes for Math 450 Continuous-time Markov chains and Stochastic Simulation." http://www. math. wustl. edu/f̃eres/Math450Lect05. pdf, Washington University in St. Louis (2007).

[7] Darren J. Wilkinson. Stochastic Modelling for Systems Biology, Chapman & Hall CRC, 2006.

[8] Chakrabarti, Deepayan, et al. "Epidemic thresholds in real networks." ACM Transactions on Information and System Security (TISSEC) 10.4 (2008): 1.

[9] Kolesnichenko, Anna, Valerio Senni, Alireza Pourranjabar, and Anne Remke. "Applying mean-field approximation to continuous time Markov chains." In Stochastic Model Checking. Rigorous Dependability Analysis Using Model Checking Techniques for Stochastic Systems, pp. 242-280. Springer Berlin Heidelberg, 2014.

[10] Norris, James R. Markov chains. No. 2. Cambridge university press, 1998.

[11] Leskovec, Jure. "Stanford network analysis package (snap)." URL http://snap. stanford. edu (2013).

[12] Kuryliak, Yulian, Michael Emmerich, and Dmytro Dosyn. "Efficient Stochastic Simulation of Network Topology Effects on the Peak Number of Infections in Epidemic Outbreaks." arXiv preprint arXiv:2202.13325 (2022).

[13] Emmerich, M., Kuryliak, Y., and Dosyn, D. (2022). Simulation of the Effects of Targeted Immunization on the Peak Number of Infections in Complex Networks. Modern Machine Learning Technologies and Data Science CEUR-WS Workshop, Leiden-Lviv, November 2022

[14] Torres, L., Chan, K. S., Tong, H., and Eliassi-Rad, T. (2021). Nonbacktracking eigenvalues under node removal: X-centrality and targeted immunization. SIAM Journal on Mathematics of Data Science, 3(2), 656-675.

[15] Costa, Guilherme S., and Silvio C. Ferreira. "Nonmassive immunization to contain spreading on complex networks." Physical Review E 101, no. 2 (2020): 022311.

[16] R. Albert, H. Jeong, and A.-L. Barabási. Attack and error tolerance of complex networks. Nature, 406: 378, 2000.

[17] Kempe, D., Kleinberg, J. and Tardos, É., 2005. Influential nodes in a diffusion model for social networks. In Automata, Languages and Programming: 32nd International Colloquium, ICALP 2005, Lisbon, Portugal, July 11-15, 2005. Proceedings 32 (pp. 1127-1138). Springer Berlin Heidelberg.

[18] Wu, P. and Pan, L., 2017. Scalable influence blocking maximization in social networks under competitive independent cascade models. Computer Networks, 123, pp.38-50.

[19] Kempe, D., Kleinberg, J. and Tardos, É., 2003, August. Maximizing the spread of influence through a social network. In Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining (pp. 137-146).

[20] Wu, L., Morstatter, F., Hu, X. and Liu, H., 2016. Mining misinformation in social media. In Big data in complex and social networks (pp. 135-162). Chapman and Hall/CRC.

[21] Kong, Z. and Yeh, E.M., 2010. Resilience to degree-dependent and cascading node failures in random geometric networks. IEEE Transactions on Information Theory, 56(11), pp.5533-5546.

[22] `https://networksciencebook.com/chapter/8`